西南交通大學
Southwest Jiaotong University

云计算技术

第12讲 云原生平台

授课教师：戴朋林

课程代码：SCAI004712

2026年6月



00 本讲内容大纲

01 Helm
应用打包 / 版本管理 / 一键部署

02 Operator 模式
CRD / Reconcile Loop / Operator SDK

03 Spark on Kubernetes
Driver Pod / Executor Pod / Spark Operator

04 GitOps
Git 唯一真实来源 / ArgoCD / Flux

05 演示代码
Helm Chart / CRD CRUD / GitOps YAML

00 本讲内容大纲

01 Helm
应用打包 / 版本管理 / 一键部署

02 Operator 模式
CRD / Reconcile Loop / Operator SDK

03 Spark on Kubernetes
Driver Pod / Executor Pod / Spark Operator

04 GitOps
Git 唯一真实来源 / ArgoCD / Flux

05 演示代码
Helm Chart / CRD CRUD / GitOps YAML

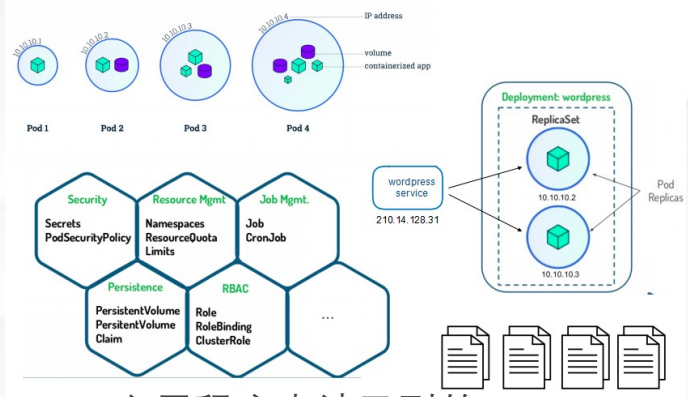
为什么需要Helm?

西南交通大学 Southwest Jiaotong University

- Kubernetes 原生对象繁多：Deployment、Service、ConfigMap、Ingress、RBAC
- 一个真实应用通常不是一个 YAML，而是一组相互关联的 YAML
- 手工维护多套环境配置时，配置漂移会快速累积

Helm Charts 帮助你定义、安装和升级即使是最复杂的 Kubernetes 应用程序。
Charts 易于创建、版本控制、共享和发布，所以开始使用 Helm，停止复制粘贴。





Wordpress应用程序中涉及到的kubernetes API对象

Helm的作用：Kubernetes的包管理器

西南交通大学 Southwest Jiaotong University

- **应用打包：**把一组 Kubernetes 资源组织成 Chart
- **参数化部署：**通过 values.yaml 适配 dev / test / prod
- **一键部署：**通过 helm install 将模板渲染后提交给 API Server
- **版本管理：**Chart 有版本，Release 也有历史版本

```
helm install myapp ./chart -f prod.yaml
```



Chart 结构：一个应用包里有什么？

西南交通大学 Southwest Jiaotong University

- Chart.yaml：描述 Chart **元信息**，例如名称、版本、依赖
- values.yaml：提供**默认参数**，模板渲染的数据来源
- charts/：存放依赖 Chart，可选
- templates/：存放 Kubernetes **资源模板**

```
myapp/
├── Chart.yaml
├── values.yaml
├── charts/
└── templates/
    ├── deployment.yaml
    ├── service.yaml
    └── ingress.yaml
```

Chart = 模板 + 参数 + 元信息

Chart 结构：Chart.yaml

西南交通大学 Southwest Jiaotong University

Chart.yaml

```
apiVersion: v2
name: myapp
description: A Helm chart for Kubernetes
type: application
version: 0.1.0
appVersion: "1.0.0"
dependencies:
  - name: redis
    version: 18.0.0
    repository: https://charts.bitnami.com/bitnami
```

- version：Chart 包自身的版本
- appVersion：应用程序的版本
- dependencies：声明依赖组件
- Chart.yaml 让应用包具备可追踪性

Chart 结构：values.yaml

西南交通大学 Southwest Jiaotong University

Values.yaml

```
replicaCount: 2  #副本数
image:  # 镜像
  repository: nginx
  tag: "1.25"
service:
  type: ClusterIP
  port: 80
resources:  #资源限制上界
  limits:
    cpu: 500m
    memory: 512Mi
```

核心思想

- values.yaml 不直接提交给 Kubernetes
- 它为 templates/ 中的占位符提供值
- 同一个 Chart 可以通过不同 values 部署到 dev / test / prod
- 参数化是 Helm 复用能力的基础

Chart 结构：templates/

```
    spec:
      serviceAccount: deis-database
      containers:
        - name: deis-database
          image:
{{ .Values.imageRegistry }}/postgres:{{ .Values.dockerTag }}
          imagePullPolicy: {{ .Values.pullPolicy }}
          ports:
            - containerPort: 5432
          env:
            - name: DATABASE_STORAGE
              value: {{ default "minio" .Values.storage }}
```

- imageRegistry: The source registry for the Docker image.
- dockerTag: The tag for the docker image.
- pullPolicy: The Kubernetes pull policy.
- storage: The storage backend, whose default is set to "minio"

模板不是最终 YAML，渲染后才是 Kubernetes 资源。

多环境部署：同一个Chart，不同Values

西南交通大学 Southwest Jiaotong University

DEV

replicas=1
image=dev
service=NodePort

TEST

replicas=2
image=rc
resource=medium

PROD

replicas=5
image=stable
ingress=enabled

一套 Chart 通过不同 values 文件渲染出不同环境的部署结果

Helmfile：多环境Helm管理

西南交通大学 Southwest Jiaotong University

- 当系统由多个 Chart 组成时，手工 helm install很混乱
- Helmfile 用一个声明式文件管理多个 Release
- 支持 environments：dev、test、prod 使用不同 values
- 适合微服务系统、平台组件和教学实验环境的批量部署

```
environments:
  dev:
    values: [values-dev.yaml]
  prod:
    values: [values-prod.yaml]
releases:
  - name: web
    chart: ./web
  - name: redis
    chart: bitnami/redis
```

Helm工作流

西南交通大学 Southwest Jiaotong University

编写Chart → 配置values → 模板渲染 → 提交API Server → 形成Release

问题	Helm的解决方法
应用包含多个K8s对象，如何整体部署?	Chart 打包
不同环境参数不一样，如何复用?	values 参数化
上线失败，如何恢复?	Release rollback
多个Chart如何批量管理?	Helmfile 声明式编排

00 本讲内容大纲

西南交通大学 Southwest Jiaotong University

01 Helm
应用打包 / 版本管理 / 一键部署

02 Operator 模式
CRD / Reconcile Loop / Operator SDK

03 Spark on Kubernetes
Driver Pod / Executor Pod / Spark Operator

04 GitOps
Git 唯一真实来源 / ArgoCD / Flux

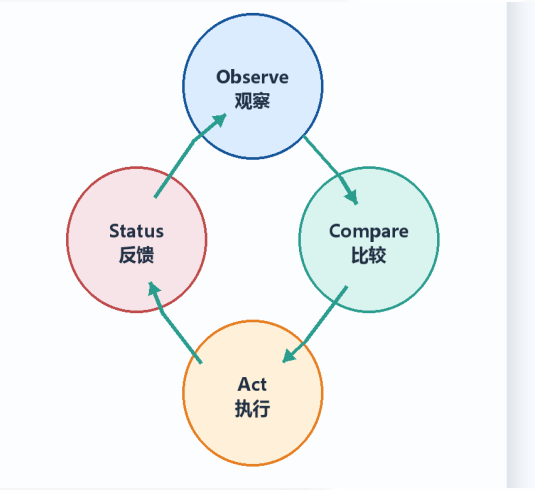
05 演示代码
Helm Chart / CRD CRUD / GitOps YAML

为什么需要 Operator?

西南交通大学 Southwest Jiaotong University

- Kubernetes 能维持通用资源的期望状态

  但不了解数据库、监控、大数据作业等领域知识
- 复杂应用需要初始化、备份、扩容、故障迁移和状态反馈
- Operator 把运维经验写成控制器，让系统持续自动收敛
- **Operator = CRD + Controller + 领域运维知识**



CRD：自定义资源

西南交通大学 Southwest Jiaotong University

- CRD 全称 Custom Resource Definition，用于扩展 Kubernetes API
- CRD 定义资源类型
- Custom Resource 创建资源实例
- 用户像操作 Deployment 一样操作业务对象
- 控制器监听这些对象，并根据 spec 推动系统状态变化

CRD 定义方式

```yaml
apiVersion: apiextensions.k8s.io/v1
kind: CustomResourceDefinition
metadata:
  name: widgets.example.com
spec:
  group: example.com
  versions:
  - name: v1
    served: true
    storage: true
  scope: Namespaced
  names:
    plural: widgets
    singular: widget
    kind: Widget
```

- group + version 组成 API 路径: example.com/v1
- kind 资源类型名称
  plural 是 kubectl 访问名
- scope 决定资源是 Namespaced 还是 Cluster 级别

Custom Resource：用户提交的期望状态

西南交通大学 Southwest Jiaotong University

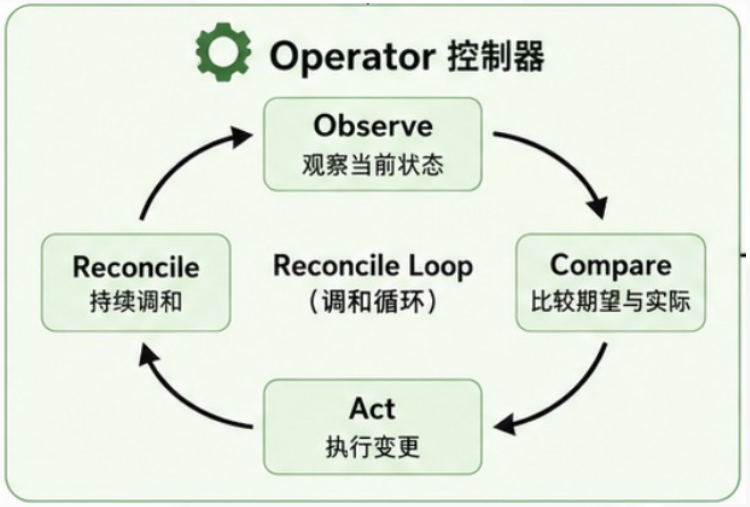
```yaml
apiVersion: example.com/v1
kind: Widget
metadata:
  name: demo-widget
spec:
  size: 3
  image: nginx:1.25
status:
  phase: Running
  readyReplicas: 3
```

- spec：用户希望系统达到的状态
- status：控制器观测到并写回的实际状态
- spec/status 分离是声明式 API 的关键设计
- “定义资源类型” V.S. “创建资源实例”

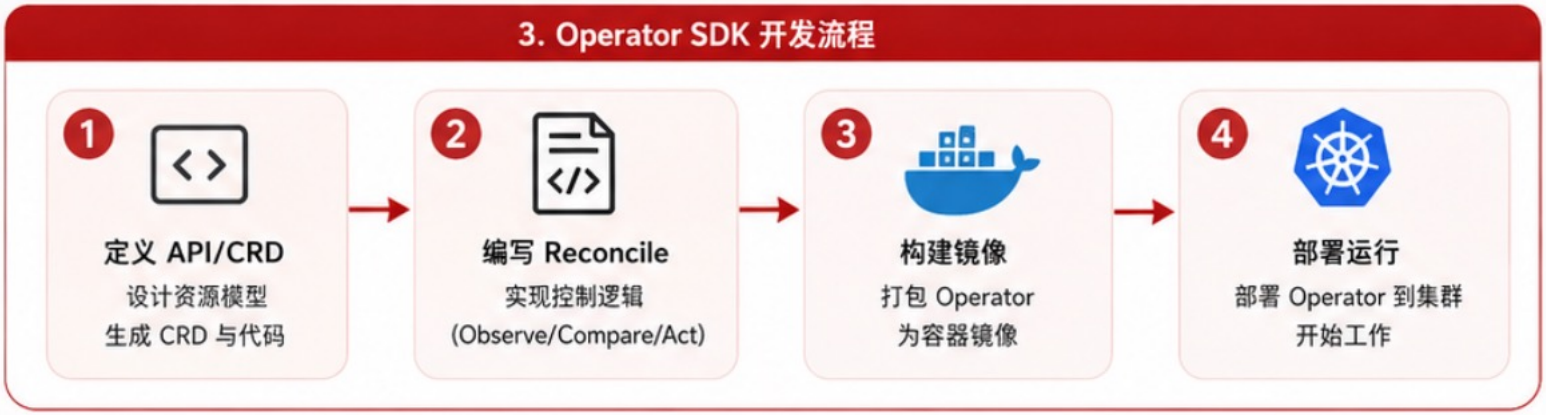
Controller：Reconcile Loop

西南交通大学 Southwest Jiaotong University

- 控制器不断比较期望状态与实际状态
- 发现差异后执行动作修正系统
- 动作完成后写回 status，并等待下一次事件或周期同步
- Controller 不应假设自己只运行一次，而要能重复执行
- 每次 Reconcile 都应尽量做到幂等：重复执行不会破坏系统



Operator SDK 开发流程




```
operator-sdk init --domain example.com --repo github.com/demo/op
operator-sdk create api --group apps --version v1 --kind WebApp
# edit api/v1/webapp_types.go
# edit controllers/webapp_controller.go
make manifests
make docker-build docker-push IMG=repo/webapp-operator:v1
make deploy IMG=repo/webapp-operator:v1
```

常见 Operator 实例

西南交通大学 Southwest Jiaotong University

Operator	管理对象	典型能力
Prometheus Operator	Prometheus / ServiceMonitor	监控系统的自动化管理
MySQL Operator	MySQLCluster / Backup	有状态应用的自动化，数据库生命周期管理
Spark Operator	SparkApplication	Driver/Executor Pod 创建、作业状态管理
Cert-Manager	Certificate / Issuer	证书申请、续期和分发

Operator 的共同点：把某类系统的“运维动作”变成控制器逻辑。

Helm vs Operator

西南交通大学 Southwest Jiaotong University

如果只是安装标准应用，优先 Helm；如果要长期管理状态，考虑 Operator

维度	Helm	Operator
核心作用	安装、升级、回滚应用包	持续运维复杂应用
执行方式	命令触发，渲染 YAML	控制器循环，持续 reconcile
状态理解	Release 历史和模板参数	业务对象 spec/status
适合场景	标准应用部署、多环境参数化	数据库、监控、大数据作业等

00 本讲内容大纲

西南交通大学 Southwest Jiaotong University

01 Helm
应用打包 / 版本管理 / 一键部署

02 Operator 模式
CRD / Reconcile Loop / Operator SDK

03 Spark on Kubernetes
Driver Pod / Executor Pod / Spark Operator

04 GitOps
Git 唯一真实来源 / ArgoCD / Flux

05 演示代码
Helm Chart / CRD CRUD / GitOps YAML

为什么需要把 Spark 跑到 Kubernetes?

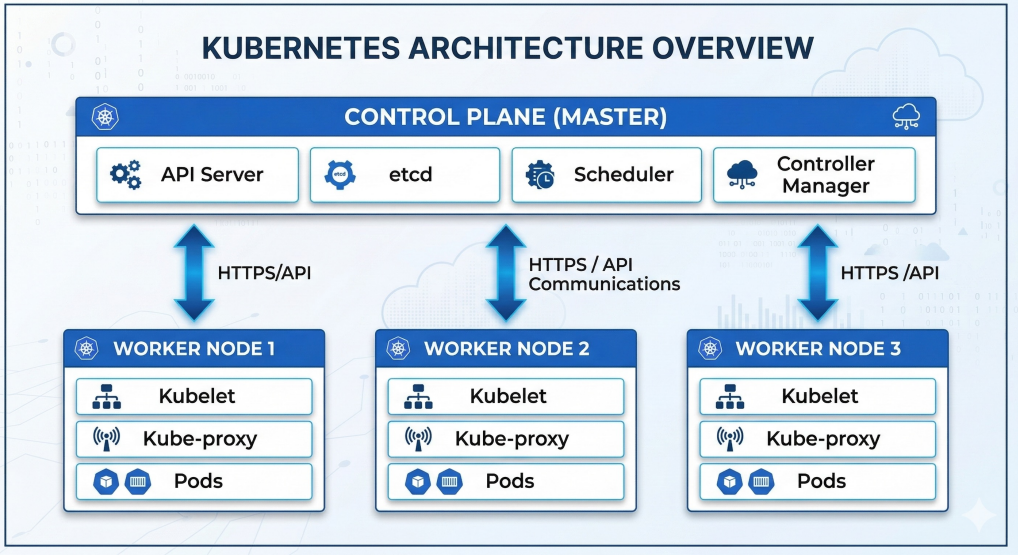
西南交通大学 Southwest Jiaotong University

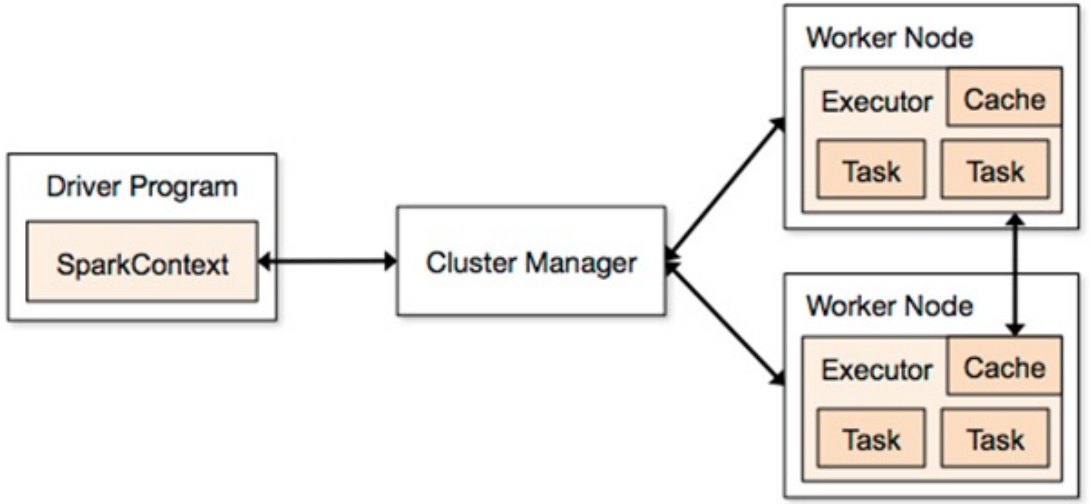
- 传统环境：Spark 运行在 Standalone 或 YARN 集群上
- 云原生环境：企业希望计算任务共享 Kubernetes 资源池
- K8S提供任务生命周期、资源隔离、镜像交付和弹性能力

Spark原生K8s支持的架构：Driver Pod + Executor Pod

西南交通大学 Southwest Jiaotong University

- Driver Pod 负责作业控制、DAG调度与任务分发
- Executor Pod 负责执行具体任务并汇报结果
- Kubernetes 负责 Pod 创建、资源分配与节点调度





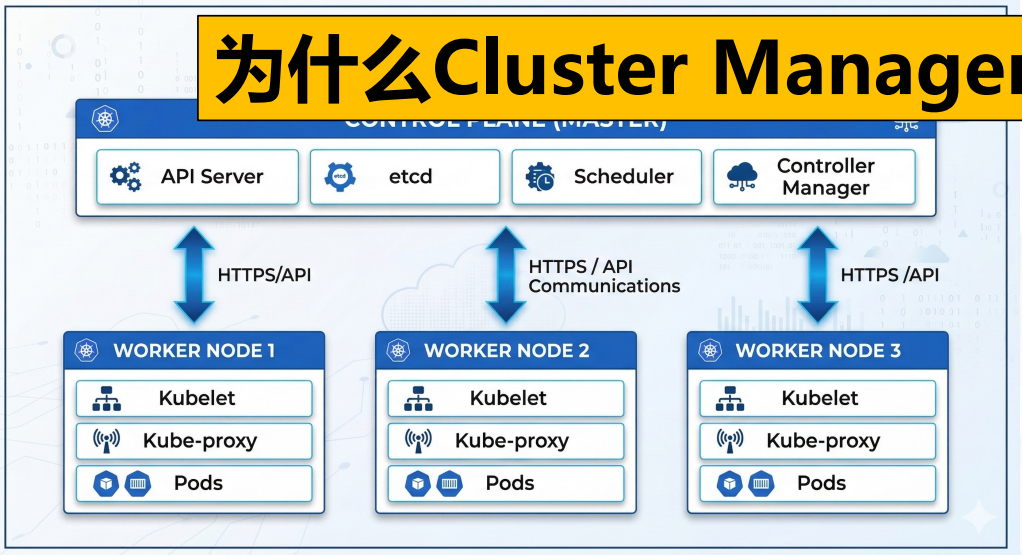
Spark on Kubernetes 让 Driver 和 Executor 以 Pod 形式运行

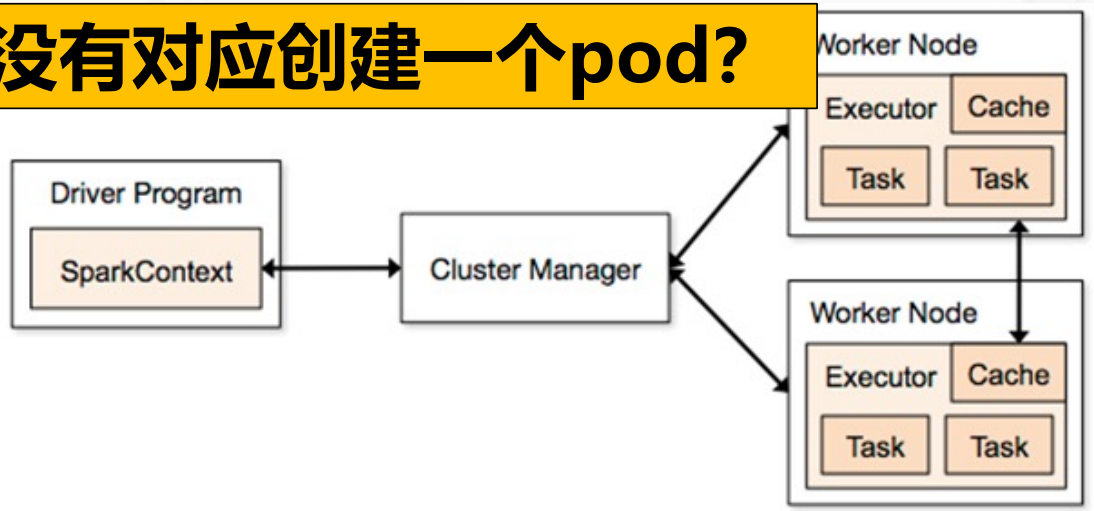
Spark原生K8s支持的架构：Driver Pod + Executor Pod

西南交通大学 Southwest Jiaotong University

- Driver Pod 负责作业控制、DAG调度与任务分发
- Executor Pod 负责执行具体任务并汇报结果
- Kubernetes 负责 Pod 创建、资源分配与节点调度

为什么Cluster Manager没有对应创建一个pod?



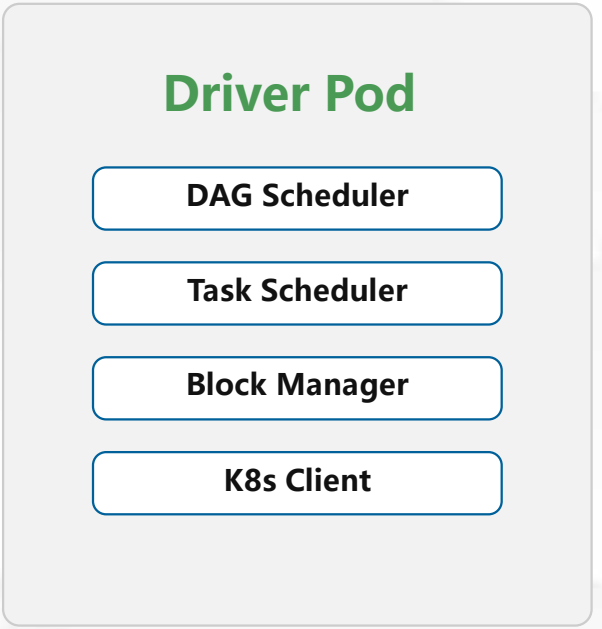


Spark on Kubernetes 让 Driver 和 Executor 以 Pod 形式运行

Driver Pod

西南交通大学 Southwest Jiaotong University

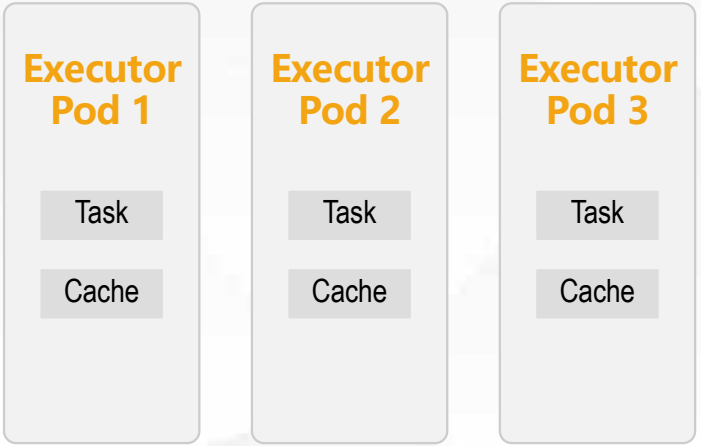
- 运行用户程序的 main 函数
- 创建 SparkContext / SparkSession
- 向 Kubernetes 请求 Executor Pod
- 把 Job 切分成 Stage 和 Task 并分发执行
- 维护任务状态、失败重试和作业生命周期



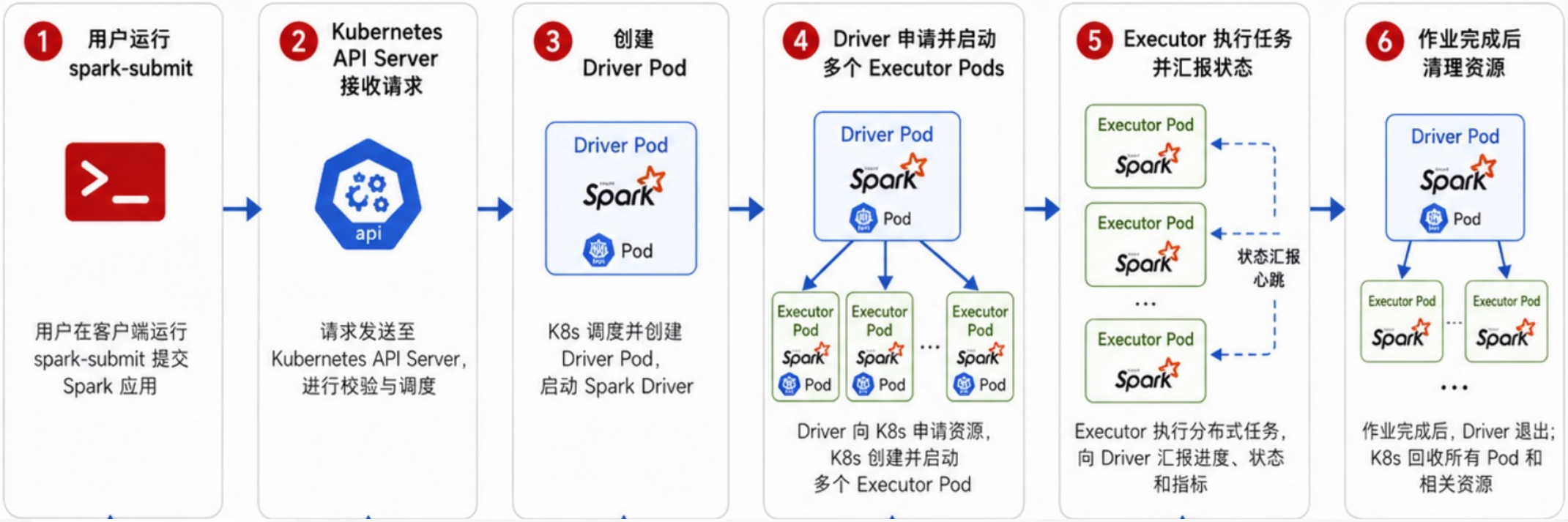
Executor Pod

西南交通大学 Southwest Jiaotong University

- 每个 Executor Pod 运行一个 Spark Executor
- Executor 内部可以用多线程运行多个 Task
- Executor 使用内存缓存数据，也可写 Shuffle 临时文件
- Executor 的生命周期由 Driver 和 Kubernetes 共同管理
- **Executor Pod 越多，并行度越强，但资源开销也越大。**



Spark Operator执行流程



SparkApplication 示例

```yaml
apiVersion: sparkoperator.k8s.io/v1beta2
kind: SparkApplication
metadata:
  name: pi
spec:
  type: Python
  mode: cluster
  image: spark:3.5.0
  mainApplicationFile: local:///opt/spark/examples/pi.py
  driver:
    cores: 1
  executor:
    instances: 3
    cores: 1
```

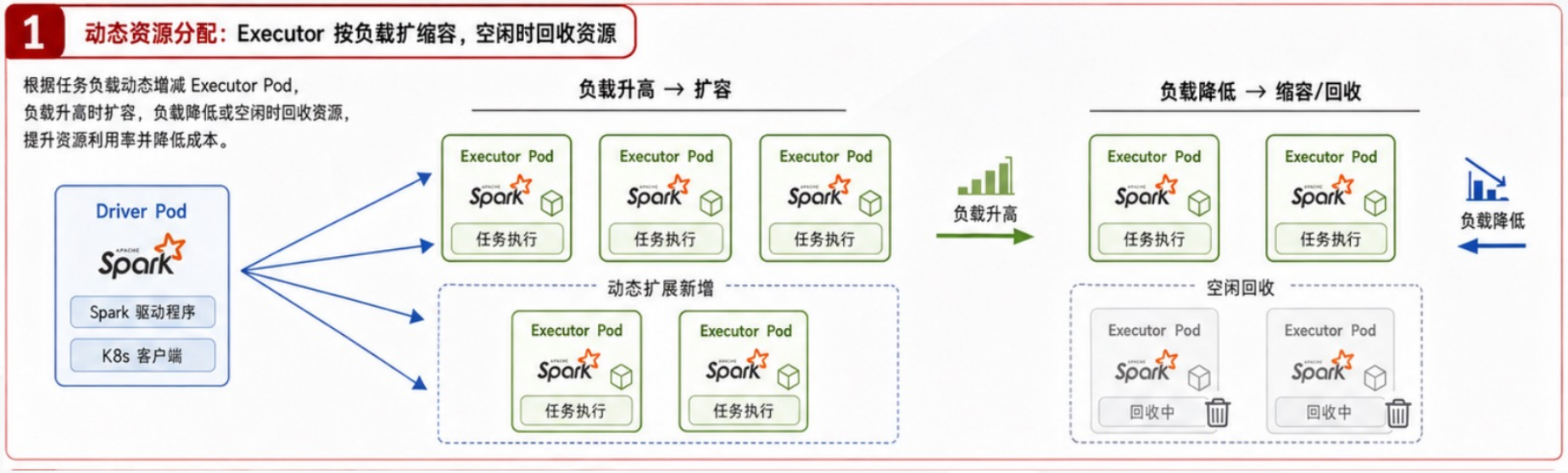
- type / mode 描述作业类型和运行模式
- image 是 Spark 与应用依赖的交付载体
- driver / executor 字段声明资源和副本数量

动态资源分配

西南交通大学 Southwest Jiaotong University

按需增加或释放Executor Pods

- Spark 作业不同阶段对 Executor 的需求不同
- 动态资源分配能显著提高资源利用率



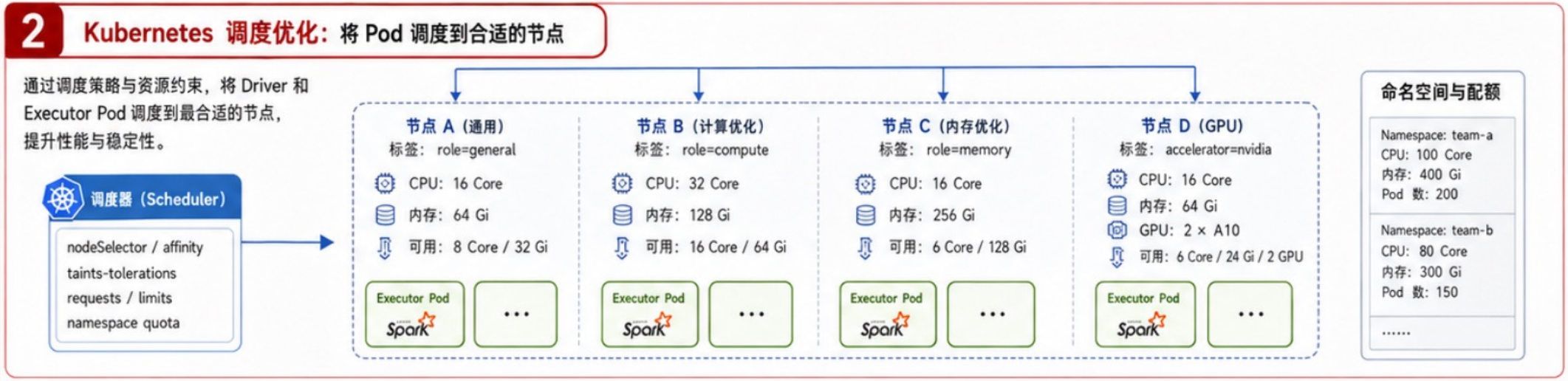
Kubernetes调度优化

西南交通大学 Southwest Jiaotong University

如何将Pod调度到合适的节点

- 调度决策pod执行的节点和对应的资源
- K8S调度器负责为新增 Executor Pod 选择节点

常见调度因素
requests / limits：决定 Pod 能否被调度
nodeSelector / affinity：把计算放到合适节点
PVC / Shuffle / IO：大数据任务常受存储路径影响
taints / tolerations：隔离特殊节点



Spark on K8s vs Spark on YARN

西南交通大学 Southwest Jiaotong University

	Spark on YARN	Spark on Kubernetes
生态基础	Hadoop 生态、HDFS、YARN 队列	容器平台、镜像、Namespace
资源隔离	YARN 队列与 Container	Namespace、ResourceQuota、Pod
部署方式	依赖 Hadoop 集群环境	依赖容器镜像与 K8s 集群
适合场景	已有 Hadoop 基础设施	云原生数据平台与统一调度





00 本讲内容大纲

西南交通大学 Southwest Jiaotong University

01 Helm
应用打包 / 版本管理 / 一键部署

02 Operator 模式
CRD / Reconcile Loop / Operator SDK

03 Spark on Kubernetes
Driver Pod / Executor Pod / Spark Operator

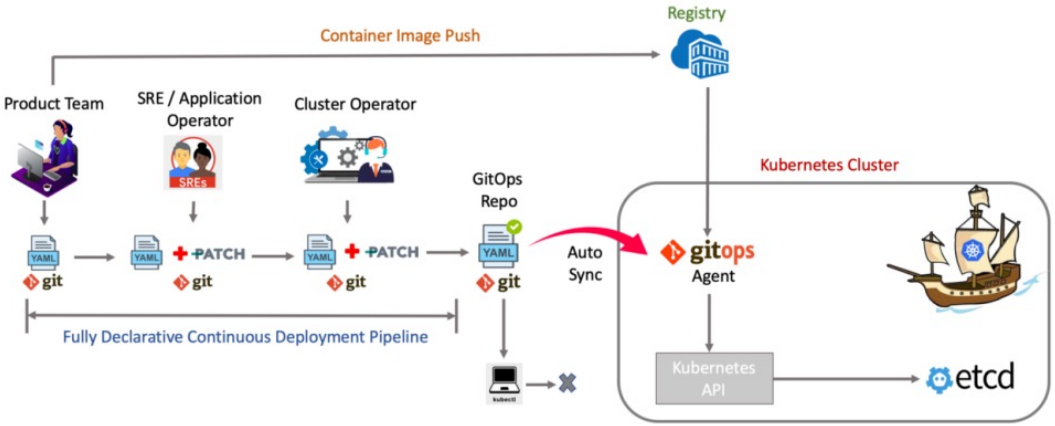
04 GitOps
Git 唯一真实来源 / ArgoCD / Flux

05 演示代码
Helm Chart / CRD CRUD / GitOps YAML

为什么需要 GitOps?

西南交通大学 Southwest Jiaotong University

- 手工 kubectl apply 容易导致线上状态不可追踪
- 多人协作时，需要知道谁改了什么、何时改、如何回滚
- Git 天然具备版本管理、审查、分支和回滚能力
- GitOps 把这些能力用于基础设施和应用部署

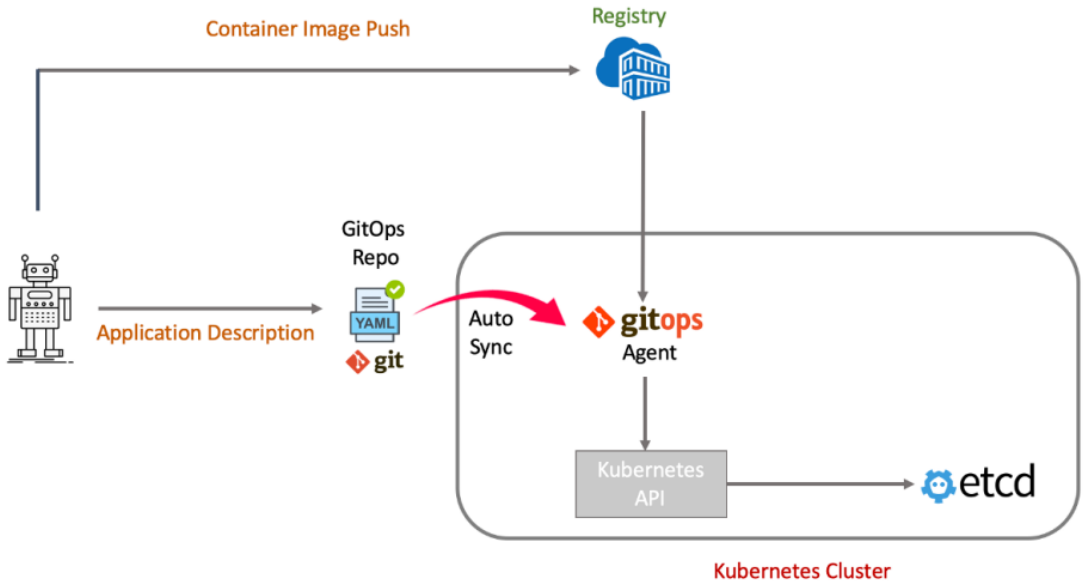


GitOps的核心理念与标准工作流

西南交通大学 Southwest Jiaotong University

Git作为唯一的真实来源

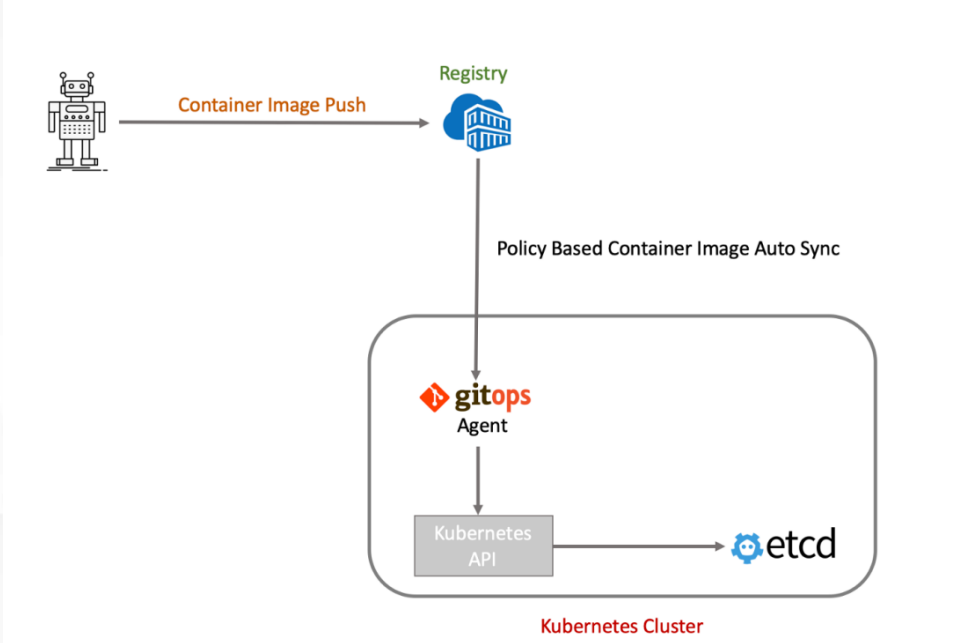
- **声明式配置**：期望状态以 YAML / Helm Chart / Kustomize 等形式保存在 Git 中
- **任何变更先进入 Git**，再由控制器同步到集群
- 集群中被手工修改的状态会被检测为 **Drift**
- **开发者主动定义声明式配置，即定义期望状态**



GitOps工作流：镜像自动更新

西南交通大学 Southwest Jiaotong University

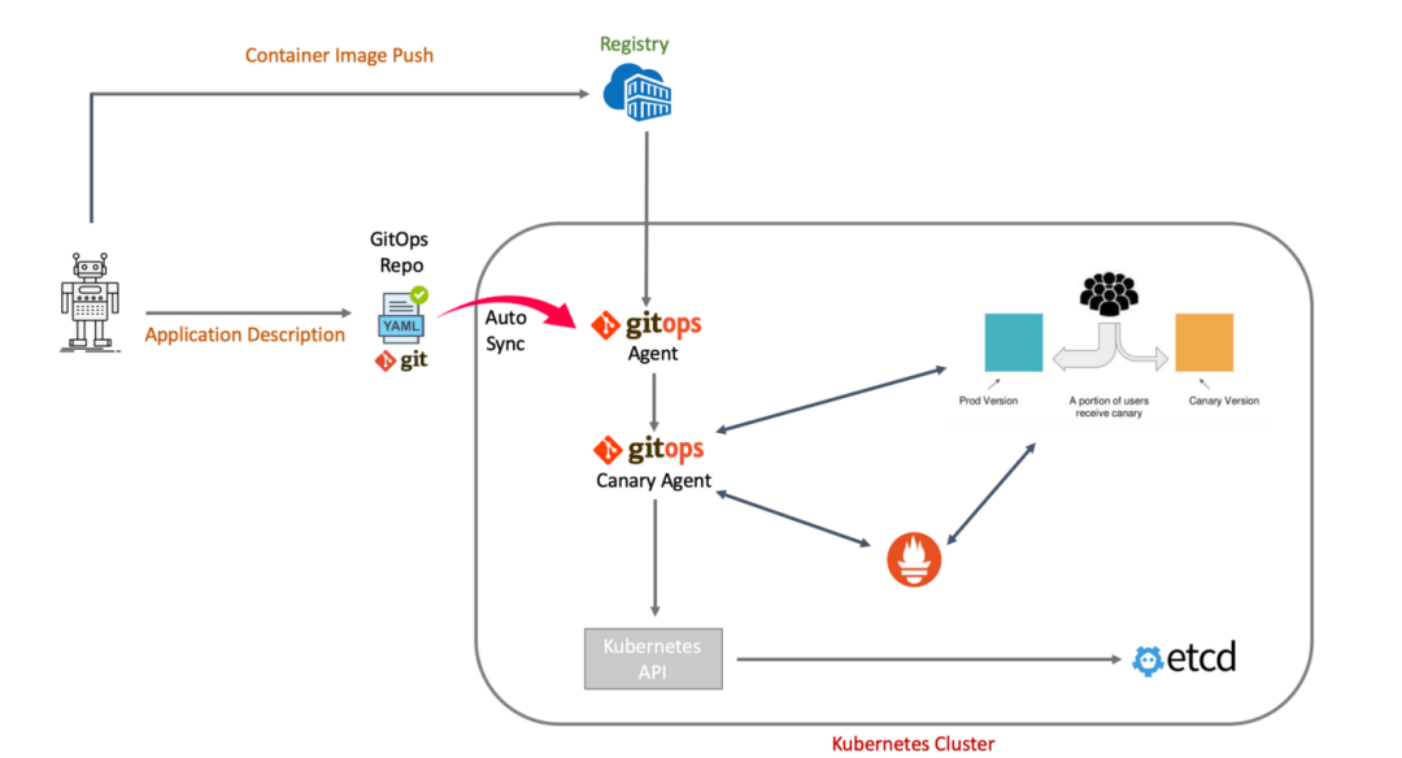
- GitOps Agent 会根据指定的策略从容器镜像仓库中自动更新新版本的容器镜像
- 例如：如果镜像有一个小版本变化，就自动更新



GitOps工作流：自动化金丝雀部署

西南交通大学 Southwest Jiaotong University

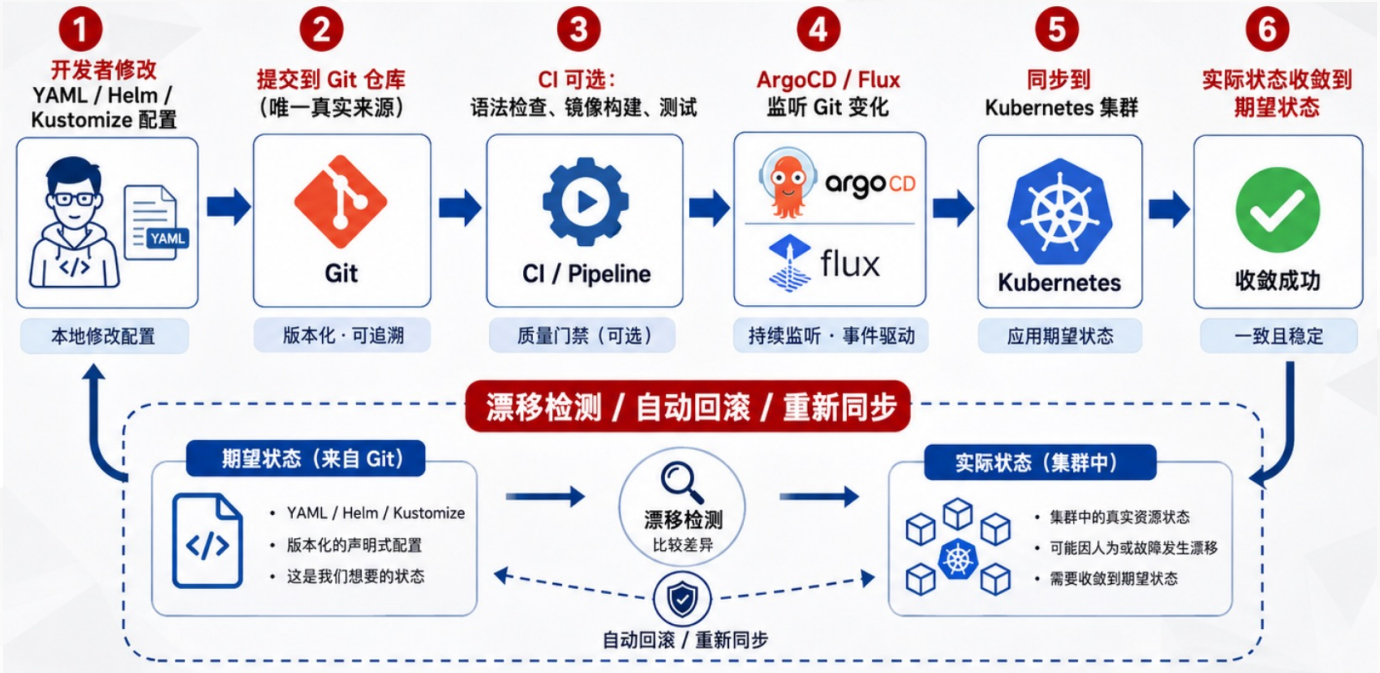
- Prometheus 测量 HTTP 请求成功率、请求平均持续时间和 Pods 健康状态等关键性能指标



ArgoCD

西南交通大学 Southwest Jiaotong University

- ArgoCD是个自动化控制工具，可以定期监听比较 Git 期望状态和集群实际状态
- 状态监听与检测：状态一致为 Synced，存在差异为 OutOfSync
- 自动同步：将期望状态应用到集群实际状态
- 漂移修复：Sync 操作把差异同步到 Kubernetes 集群



Flux

西南交通大学 Southwest Jiaotong University

- Flux也是一个GitOps自动化控制工具，为K8S集群的持续交付而设计
- 由多个控制器组成
  - source-controller：获取 Git / Helm 源
  - kustomize-controller：应用 Kustomize 配置
  - helm-controller：管理 Helm Release
  - notification-controller：事件通知
- Flux是Git原生，原生支持与Kustomize/Helm集成



00 本讲内容大纲

西南交通大学 Southwest Jiaotong University

01 Helm
应用打包 / 版本管理 / 一键部署

02 Operator 模式
CRD / Reconcile Loop / Operator SDK

03 Spark on Kubernetes
Driver Pod / Executor Pod / Spark Operator

04 GitOps
Git 唯一真实来源 / ArgoCD / Flux

05 演示代码
Helm Chart / CRD CRUD / GitOps YAML

Demo 1：Python 生成 Helm Chart 目录结构

西南交通大学 Southwest Jiaotong University

```python
# create_helm_chart.py
from pathlib import Path
root = Path("web-chart")
(root / "templates").mkdir(parents=True, exist_ok=True)
(root / "Chart.yaml").write_text("""apiVersion: v2
name: web-chart
version: 0.1.0
""")
(root / "values.yaml").write_text("""replicaCount: 2
image:
  repository: nginx
  tag: "1.25"
""")
tpl = """apiVersion: apps/v1
kind: Deployment
spec:
  replicas: {{ .Values.replicaCount }}
"""
(root / "templates" / "deployment.yaml").write_text(tpl)
```

- Chart 不是抽象概念，而是一组目录和文件

Demo 2：生成 Deployment 模板文件

西南交通大学 Southwest Jiaotong University

```
deployment = r'''
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}
spec:
  replicas: {{ .Values.replicaCount }}
  template:
    spec:
      containers:
      - name: web
        image:
{{ .Values.image.repository }}:{{ .Values.image.tag }}
'''
(chart / 'templates' /
'deployment.yaml').write_text(deployment)
```

- 注意.Values 如何把参数注入模板
- 模板不是最终 YAML
- 渲染后才提交给 API Server
- 可增加 helm lint / helm template 检查步骤

Demo 3：Kubernetes Python 客户端创建 CRD

西南交通大学 Southwest Jiaotong University

```
from kubernetes import client, config
config.load_kube_config()
ext = client.ApiextensionsV1Api()


crd = {
  'apiVersion':'apiextensions.k8s.io/v1',
  'kind':'CustomResourceDefinition',
  'metadata': {'name':'demos.example.com'},
  'spec': {'group':'example.com', 'scope':'Namespaced', ...}
}
ext.create_custom_resource_definition(crd)
```

Demo 4：自定义资源 CRUD

```
api = client.CustomObjectsApi()
body = {
  'apiVersion':'example.com/v1',
  'kind':'Demo',
  'metadata': {'name':'hello'},
  'spec': {'message':'cloud native'}
}
api.create_namespaced_custom_object('example.com','v1','default','demos',body)
api.get_namespaced_custom_object('example.com','v1','default','demos','hello')
api.patch_namespaced_custom_object('example.com','v1','default','demos','hello',
{'spec':{'message':'updated'}})
api.delete_namespaced_custom_object('example.com','v1','default','demos','hello')
```

- CRD 是类型，CR 是实例
- 控制器存在时才会根据 CR 执行自动化动作

Demo 5：GitOps 工作流 YAML 配置

西南交通大学 Southwest Jiaotong University

```yaml
apiVersion: argoproj.io/v1alpha1
kind: Application
metadata:
  name: webapp
  namespace: argocd
spec:
  project: default
  source:
    repoURL: https://github.com/example/cloud-native-demo.git
    targetRevision: main
    path: manifests/webapp
  destination:
    server: https://kubernetes.default.svc
    namespace: demo
  syncPolicy:
    automated:
      prune: true
      selfHeal: true
```

- Application 把 Git 路径和目标集群绑定起来
- prune 删除 Git 中已移除的资源
- selfHeal 发现漂移后自动纠偏

课堂思考题

西南交通大学 Southwest Jiaotong University

- Helm 为什么不能完全替代 Operator?
- CRD 中 spec 和 status 分离有什么好处?
- Spark on Kubernetes 相比 Spark on YARN 的核心变化是什么?
- GitOps 为什么强调 Git 是唯一真实来源?
- 如果 Git 中配置错误，自动同步会带来什么风险？如何治理?